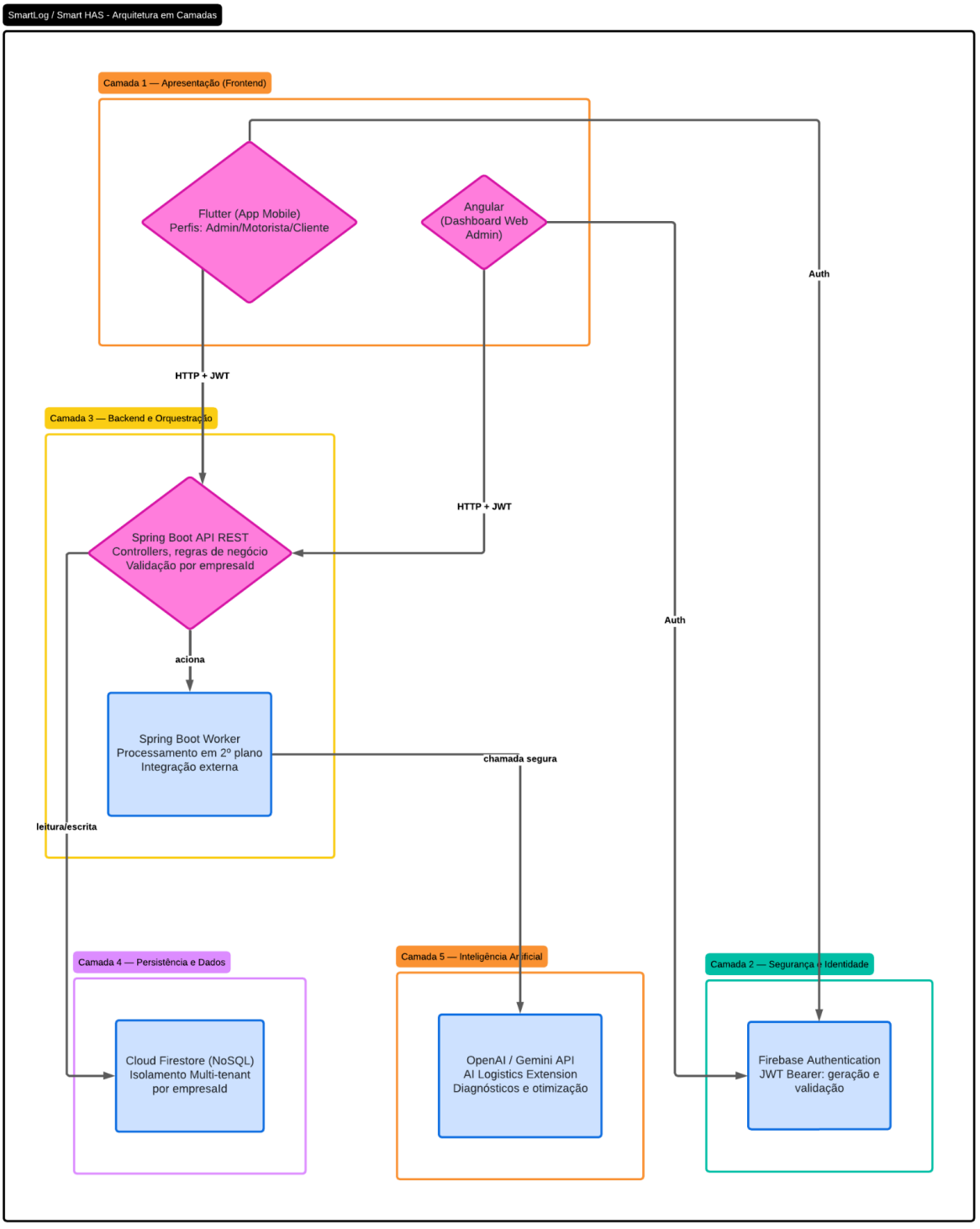
A02 — Definir arquitetura da solução



2. Responsabilidades das Camadas

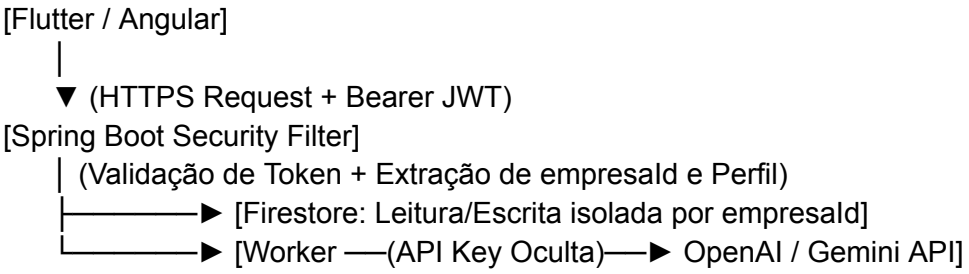
- **Camada de Apresentação (Flutter & Angular)**:
    - **Flutter (Mobile)**: Interface reativa destinada a Administradores de campo, Motoristas e Clientes finais. Utiliza gerenciamento de estado via Provider (`ChangeNotifier`).
    - **Angular (Web Dashboard)**: Painel administrativo corporativo voltado para auditoria operacional, gestão global de rotas e acompanhamento de métricas em telas de alta resolução.
- **Camada de Identidade e Segurança (Firebase Authentication)**:
    - Centraliza o ciclo de vida de credenciais, validação de acessos e emissão de tokens JWT seguros para os perfis `ADMIN`, `MOTORISTA` e `CLIENTE`.
- **Camada de Backend e Orquestração (Spring Boot & Worker)**:
    - **Spring Boot API REST**: Expõe endpoints padronizados em JSON, valida tokens recebidos no cabeçalho `Authorization: Bearer` e aplica controle de acesso baseado em perfis (RBAC).
    - **Spring Boot Worker**: Processa tarefas em segundo plano e realiza chamadas para o motor de IA generativa de forma isolada.
- **Camada de Persistência (Cloud Firestore)**:
    - Banco de dados NoSQL documental de baixa latência encarregado de armazenar registros operacionais (entregas, frotas, motoristas e empresas).
- **Camada de Inteligência Artificial (AI Logistics Extension - OpenAI / Gemini API)**:
    - Fornece diagnósticos preditivos de gargalos em entregas, análises operacionais por região e sugestões de otimização de rotas logísticas.

3. Justificativa Técnica para Manutenção do Cloud Firestore

A decisão de manter o **Cloud Firestore** fundamenta-se nos seguintes pontos técnicos:

1. **Sincronização em Tempo Real (*Real-time Streams*)**: A arquitetura baseada em documentos e WebSockets/Streams permite que alterações nos status de entregas feitas pelo motorista ou administrador reflitam instantaneamente na interface sem necessidade de *polling* HTTP contínuo.
2. **Isolamento Multi-Tenant por `empresaId`**: A estrutura de coleções do Firestore viabiliza a separação lógica e segura dos dados de cada empresa parceira de forma escalável e nativa.
3. **Escalabilidade Serverless**: Baixo custo operacional de manutenção e alta disponibilidade integrada diretamente aos serviços de autenticação do Google Cloud/Firebase.

4. Integrações e Limites de Segurança



Autenticação e Trânsito de Dados:

- Todas as requisições HTTP enviadas pelo Flutter e Angular para a API Spring Boot exigem o cabeçalho `Authorization: Bearer <FIREBASE_ID_TOKEN>`.

Isolamento Contextual e RBAC:

- O backend valida as *claims* do token antes de executar qualquer operação. Requisições com permissões insuficientes retornam `403 Forbidden` e tokens inválidos/ausentes retornam `401 Unauthorized`.
- Todas as consultas e gravações no Firestore são filtradas pelo `empresaId` do usuário logado.

Proteção de Credenciais:

- Chaves de API externas (OpenAI/Gemini e Service Accounts do Firebase Admin) ficam restritas ao backend em variáveis de ambiente (`.env`), impedindo a exposição de dados sensíveis nas aplicações frontend.